name, and last name (the roles are hardcoded in the controller for now). The following code shows how to do that:

```
@Test
void whenAuthenticatedThenReturnUser() {
  var expectedUser = new User("jon.snow", "Jon", "Snow",
    List.of("employee", "customer"));                          ❶

  webClient
    .mutateWith(configureMockOidcLogin(expectedUser))          ❷
    .get()
    .uri("/user")
    .exchange()
    .expectStatus().is2xxSuccessful()
    .expectBody(User.class)                                    ❸
    .value(user -> assertThat(user).isEqualTo(expectedUser));
}

private SecurityMockServerConfigurers.OidcLoginMutator
➥ configureMockOidcLogin(User expectedUser) {
  return SecurityMockServerConfigurers.mockOidcLogin().idToken(
   builder -> {                                                ❹
      builder.claim(StandardClaimNames.PREFERRED_USERNAME,
        expectedUser.username());
      builder.claim(StandardClaimNames.GIVEN_NAME,
        expectedUser.firstName());
      builder.claim(StandardClaimNames.FAMILY_NAME,
        expectedUser.lastName());
    });
}
```

❶ The expected authenticated user

❷ Defines an authentication context based on OIDC and uses the expected user

❸ Expects a User object with the same information as the currently authenticated user

❹ Builds a mock ID Token

Finally, run the tests as follows:

```
$ ./gradlew test --tests UserControllerTests
```

The testing utilities supplied by Spring Security cover a wide array of scenarios and integrate well with `WebTestClient`. In the next section,